

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN II
MODUL 17
“SKEMA PEMROSESAN SEKUENSIAL”



DISUSUN OLEH:
BENING PUTRI NARESWARI SUKARNO
109082500211

S1 IF-13-02

DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng

Asisten Praktikum
Muhammad Agha Zulfadhli
Renisa Assyifa Putri

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026

DASAR TEORI

Algoritma dapat diartikan sebagai serangkaian langkah yang disusun secara teratur untuk menyelesaikan suatu masalah atau mencapai tujuan tertentu. Dalam pemrograman, algoritma menjadi pedoman dalam pembuatan program karena menjelaskan proses yang harus dilakukan agar data yang dimasukkan dapat diolah dan menghasilkan keluaran yang sesuai. Salah satu bentuk algoritma yang paling sederhana adalah struktur sekuensial, yaitu proses yang dijalankan secara berurutan dari langkah awal hingga langkah akhir tanpa adanya percabangan atau pengulangan. Struktur ini biasanya digunakan pada program-program dasar yang memiliki alur pengerjaan sederhana dan mudah dipahami.

Selain itu, dalam pemrograman juga dikenal konsep pengulangan (looping), yaitu proses menjalankan perintah yang sama secara berulang sesuai kebutuhan. Penggunaan pengulangan membuat program menjadi lebih efisien karena tidak perlu menuliskan instruksi yang sama berkali-kali. Pada proses ini sering digunakan variabel pencacah (counter) untuk menghitung banyaknya pengulangan yang terjadi dan akumulator (accumulator) untuk menyimpan hasil perhitungan yang terus bertambah selama program berjalan. Kombinasi konsep-konsep tersebut banyak diterapkan dalam berbagai kasus, seperti pengolahan data, simulasi sederhana, maupun perhitungan matematis yang memerlukan banyak proses berulang.

SOAL LATIHAN MODUL 17

1. Soal 1

Source Code:

```
package main

import "fmt"

func main() {

    var x float64
    var jumlah float64
    var rata float64
    var cacah int

    fmt.Scan(&x)

    for x != 9999 {

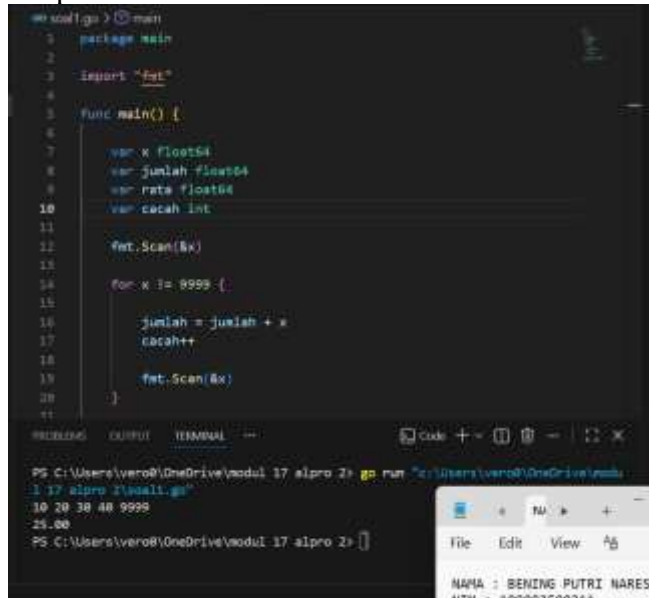
        jumlah = jumlah + x
        cacah++

        fmt.Scan(&x)
    }

    rata = jumlah / float64(cacah)

    fmt.Printf("%.2f\n", rata)
}
```

Output:



The screenshot shows a Go program in a text editor and its execution in a terminal. The program is a simple loop that reads numbers until it encounters '9999', calculates the sum, and then the average. The terminal output shows the input sequence '10 20 30 40 9999' and the resulting average '25.00'.

```
1 package main
2
3 import "fmt"
4
5 func main() {
6
7     var x float64
8     var jumlah float64
9     var rata float64
10    var cacah int
11
12    fmt.Scan(&x)
13
14    for x != 9999 {
15
16        jumlah = jumlah + x
17        cacah++
18
19        fmt.Scan(&x)
20    }
21}
```

PS C:\Users\vero\OneDrive\modul 17 alpro 2> go run "c:\Users\vero\OneDrive\modul 17 alpro 2\modul17.go"

10 20 30 40 9999

25.00

PS C:\Users\vero\OneDrive\modul 17 alpro 2> |

Deskripsi Program:

Program ini berfungsi untuk menghitung rata-rata dari beberapa bilangan real. Pengguna dapat memasukkan data sebanyak yang diinginkan dan mengakhiri input dengan angka 9999 sebagai marker. Program kemudian menjumlahkan seluruh data yang valid, menghitung jumlah data, dan menampilkan nilai rata-ratanya.

Contoh input:

10
20
30
40
9999

Angka 10, 20, 30, dan 40 adalah data yang akan dihitung. Angka 9999 berfungsi sebagai penanda bahwa input telah selesai dan tidak ikut dihitung.

Jumlah data = $10 + 20 + 30 + 40 = 100$

Banyak data = 4

Rata-rata = $100 / 4 = 25$

Jadi, outputnya:

Rata-rata = 25.00

2. Soal 2

Source code (jika dijadikan 1 syntax:

```
package main

import "fmt"

func main() {

    var x, kata string
    var n int

    // input string yang dicari
    fmt.Scanln(&x)

    // banyak data
    fmt.Scanln(&n)

    jumlah := 0
    posisi := -1

    for i := 1; i <= n; i++ {

        fmt.Scanln(&kata)

        if kata == x {

            jumlah++

            // simpan posisi pertama
            if posisi == -1 {
                posisi = i
            }
        }
    }

    // a. apakah ada
    if jumlah > 0 {
```

```
        fmt.Println("Ada")
    } else {
        fmt.Println("Tidak Ada")
    }

    // b. posisi pertama
    fmt.Println(posisi)

    // c. jumlah kemunculan
    fmt.Println(jumlah)

    // d. minimal 2 kali
    if jumlah >= 2 {
        fmt.Println("Ya")
    } else {
        fmt.Println("Tidak")
    }
}
```

```
soal2.go > ...
5 func main() {
34 // a. apakah ada
35 if jumlah > 0 {
36     fmt.Println("Ada")
37 } else {
38     fmt.Println("Tidak Ada")
39 }
40
41 // b. posisi pertama
42 fmt.Println(posisi)
43
44 // c. jumlah kemunculan
45 fmt.Println(jumlah)
46
47 // d. minimal 2 kali
48 }

PROBLEMS 2 OUTPUT TERMINAL ...
1 17 alpro 2\soal2.go"
melon
5
semangka
pisang
melon
nanas
kiwi
Ada
3
1
Tidak
PS C:\Users\vero0\OneDrive\modul 17 alpro 2>
```

Diberikan string x dan n buah string. x adalah data pertama yang dibaca, n adalah data bilangan yang dibaca kedua, dan n data berikutnya adalah data string. Buat algoritma untuk menjawab pertanyaan berikut:

- a. Menentukan apakah x ada

```
input x

input n

ada <- false

for i <- 1 sampai n do

    input kata

    if kata = x then
```

```
        ada <- true
    endif
endfor

output ada
```

Deskripsi:

Program digunakan untuk menentukan apakah suatu string x terdapat di dalam kumpulan n string yang diberikan. Program membaca setiap string dan membandingkannya dengan string x. Jika ditemukan string yang sama, maka variabel ada akan bernilai true.

b. Menentukan posisi pertama x ditemukan

```
input x
input n

posisi <- -1

for i <- 1 sampai n do
    input kata

    if kata = x dan posisi = -1 then
        posisi <- i
    endif
endfor

output posisi
```

Deskripsi:

Program digunakan untuk mencari posisi pertama kemunculan string x pada kumpulan data.

c. Menghitung jumlah kemunculan x

```
input x

input n

jumlah <- 0

for i <- 1 sampai n do
    input kata

    if kata = x then
        jumlah <- jumlah + 1
    endif
endfor

output jumlah
```

Deskripsi:

Program digunakan untuk menghitung jumlah kemunculan string x dalam kumpulan data.

d. Menentukan apakah x muncul minimal 2 kali

```
input x

input n

jumlah <- 0

for i <- 1 sampai n do
    input kata
```

```
        if kata = x then
            jumlah <- jumlah + 1
        endif
    endfor

    if jumlah >= 2 then
        output "YA"
    else
        output "TIDAK"
    endif
```

Deskripsi:

Program digunakan untuk menentukan apakah string x muncul minimal dua kali dalam kumpulan data.

3. Soal 3

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
)

func main() {

    var n int

    // banyak titik hujan
    fmt.Scan(&n)

    var A, B, C, D int

    for i := 0; i < n; i++ {

        // menghasilkan angka acak 0 sampai 1
        x := rand.Float64()
        y := rand.Float64()

        // daerah A (kiri bawah)
        if x < 0.5 && y < 0.5 {
```

```
        A++

        // daerah B (kanan bawah)
    } else if x >= 0.5 && y < 0.5 {

        B++

        // daerah C (kanan atas)
    } else if x >= 0.5 && y >= 0.5 {

        C++

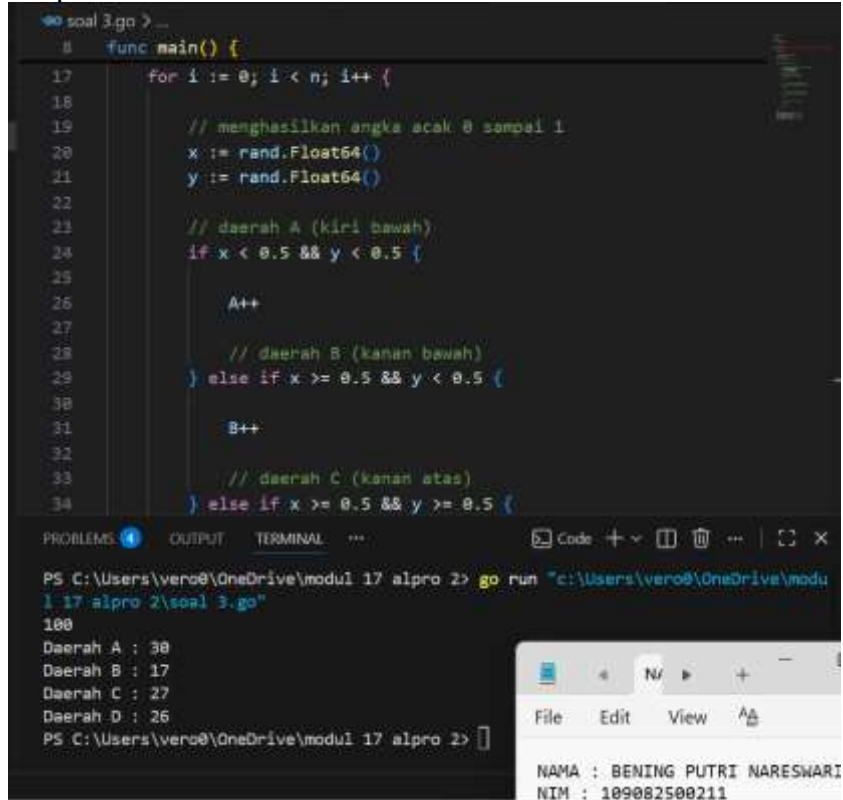
        // daerah D (kiri atas)
    } else {

        D++

    }
}

fmt.Println("Daerah A :", A)
fmt.Println("Daerah B :", B)
fmt.Println("Daerah C :", C)
fmt.Println("Daerah D :", D)
}
```

Output:



```
soal3.go > ...
func main() {
    for i := 0; i < n; i++ {
        // menghasilkan angka acak 0 sampai 1
        x := rand.Float64()
        y := rand.Float64()

        // daerah A (kiri bawah)
        if x < 0.5 && y < 0.5 {
            A++
        } else if x >= 0.5 && y < 0.5 {
            B++
        } else if x >= 0.5 && y >= 0.5 {
            C++
        } else {
            D++
        }
    }
}
```

PS C:\Users\vero0\OneDrive\modul 17 alpro 2> go run "c:\Users\vero0\OneDrive\modul 17 alpro 2\soal 3.go"

100
Daerah A : 30
Daerah B : 17
Daerah C : 27
Daerah D : 26
PS C:\Users\vero0\OneDrive\modul 17 alpro 2>

NAMA : BENING PUTRI NARESWARI
NIM : 109082500211

Deskripsi Program:

Program ini digunakan untuk mensimulasikan curah hujan pada sebuah wilayah berbentuk persegi yang dibagi menjadi empat daerah, yaitu A, B, C, dan D. Setiap tetes hujan direpresentasikan sebagai sebuah titik dengan koordinat acak (x,y) yang nilainya berada pada rentang 0 sampai 1. Program akan membangkitkan sejumlah titik hujan sesuai banyaknya data yang dimasukkan pengguna. Setiap titik kemudian diperiksa posisinya untuk menentukan berada pada daerah A, B, C, atau D. Setelah seluruh titik selesai diproses, program menghitung banyaknya titik hujan pada setiap daerah dan menampilkan hasil simulasi curah hujan pada masing-masing daerah.

Alurnya;;

- Program dimulai dengan membaca jumlah titik hujan yang akan disimulasikan. Selanjutnya program membangkitkan koordinat x dan y secara acak menggunakan fungsi random.
- Setiap koordinat yang terbentuk akan diperiksa letaknya berdasarkan nilai x dan y.
- Jika titik berada di bagian kiri bawah maka dihitung sebagai daerah A, jika berada di kanan bawah dihitung sebagai daerah B, jika berada di kanan atas dihitung sebagai daerah C, dan jika berada di kiri atas dihitung sebagai daerah D.
- Setelah semua titik selesai diproses, program menghitung jumlah titik pada masing-masing daerah dan menampilkan hasilnya sebagai simulasi curah hujan.

Contoh input dan outputnya:

100

Daerah A : 30

Daerah B : 17

Daerah **C** : 27

Daerah **D** : 26

Sebanyak 100 titik hujan yang dibangkitkan secara acak. Setelah seluruh titik diproses, diperoleh 30 titik berada pada daerah A, 17 titik berada pada daerah B, 27 titik berada pada daerah C, dan 26 titik berada pada daerah D. Jumlah seluruh titik yang terhitung adalah 100 titik. Perbedaan jumlah titik pada setiap daerah terjadi karena koordinat dibangkitkan secara acak menggunakan fungsi `rand.Float64()`, sehingga hasil yang diperoleh dapat berbeda setiap kali program dijalankan.

4. Soal 4

Source Code:

```
package main

import "fmt"

func main() {

    var n int

    var jumlah float64

    fmt.Scan(&n)

    for i := 1; i <= n; i++ {

        // hitg penyebut: 1,3,5,7,...
        penyebut := float64(2*i - 1)

        // nilai suku
        suku := 1.0 / penyebut

        // jika suku ke-i genap, ubah menjadi negatif
        if i%2 == 0 {
            suku = -suku
        }
    }
}
```

```
        jumlah = jumlah + suku

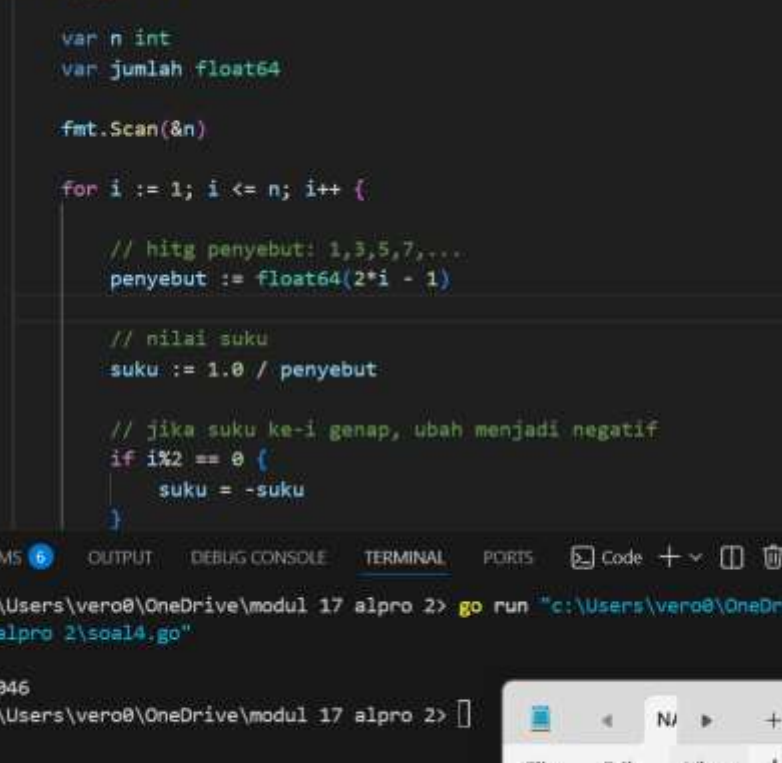
    }

    pi := 4 * jumlah

    fmt.Printf("%.6f\n", pi)

}
```

Output:



```
soal4.go > main
5 func main() {
6
7     var n int
8     var jumlah float64
9
10    fmt.Scan(&n)
11
12    for i := 1; i <= n; i++ {
13
14        // hitg penyebut: 1,3,5,7,...
15        penyebut := float64(2*i - 1)
16
17        // nilai suku
18        suku := 1.0 / penyebut
19
20        // jika suku ke-i genap, ubah menjadi negatif
21        if i%2 == 0 {
22            suku = -suku
23        }
24    }
25
26    jumlah += suku
27 }
28
29 PS C:\Users\vero0\OneDrive\modul 17 alpro 2> go run "c:\Users\vero0\OneDrive\modul 17 alpro 2\soal4.go"
6
2.976046
PS C:\Users\vero0\OneDrive\modul 17 alpro 2>
```

Deskripsi Program:

Program ini digunakan untuk menghitung nilai π (phi) menggunakan deret Leibniz. Program akan menjumlahkan suku-suku deret secara bergantian antara positif dan negatif. Semakin banyak suku yang dihitung, hasil pendekatan nilai π akan semakin mendekati nilai sebenarnya. Pada program ini, setiap suku dihitung satu per satu kemudian ditambahkan ke dalam total penjumlahan hingga mencapai jumlah iterasi yang ditentukan pengguna.

- a) Program membaca banyaknya suku yang akan dihitung. Selanjutnya program melakukan perulangan dari suku pertama hingga suku ke-n.
- b) Pada setiap perulangan, program menghitung nilai suku sesuai rumus Leibniz.
- c) Jika nomor suku ganjil maka nilainya positif, sedangkan jika nomor suku genap maka nilainya negatif.
- d) Semua suku dijumlahkan ke dalam sebuah variabel penampung.
- e) Setelah seluruh suku selesai dihitung, hasil penjumlahan dikalikan 4 untuk memperoleh pendekatan nilai π . Hasil akhirnya kemudian ditampilkan ke layar.

5. Soal 5

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
)

func main() {

    var n int

    var toppingPizza int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {

        x := rand.Float64()

        y := rand.Float64()

        // cekkk ap titik berada di dalam pizza
        if (x-0.5)*(x-0.5)+(y-0.5)*(y-0.5) <= 0.25 {

            toppingPizza++

        }
    }
}
```


Alur :

Membaca banyak topping (n).

1. Membuat titik acak (x,y) sebanyak n.
2. Menghitung jarak titik terhadap pusat pizza (0.5,0.5).
3. Jika memenuhi:

$$(x - 0.5)^2 + (y - 0.5)^2 \leq 0.25$$

maka topping berada di dalam pizza.

4. Hitung banyak topping yang berada di pizza.
5. Hitung nilai π dengan rumus:

$$\pi = 4 \times \frac{\text{Topping pada Pizza}}{\text{Banyak Topping}}$$

6. Tampilkan hasil.

DAFTAR PUSTAKA

MASUK PTN. (2025). *Struktur urutan, percabangan, dan perulangan dalam pemrograman*.
<https://masuk-ptn.com/materi/algorithmadanpemrogramandasarmateriinformatikakelas-10/struktururutanpercabanganperulangan>